

Delta Lake & Databricks Jobs

Hands-On Lab Guide

Copy-paste exercises with expected output, built for Databricks Free / Community Edition
Companion to: Delta Lake & Databricks Jobs — A Practical Deep-Dive Guide

Table of Contents

- 0. Before You Begin 3
- Lab 1 — ACID Transactions in Practice..... 3
 - 1.1 Create Your First Delta Table..... 3
 - 1.2 Look Inside the Transaction Log 4
 - 1.3 Update the Table and Watch a New Commit Appear..... 4
 - 1.4 See Atomicity Protect You From a Failed Write 4
- Lab 2 — Time Travel in Practice..... 5
 - 2.1 Build Up Some Version History..... 5
 - 2.2 Query Past Versions..... 5
 - 2.3 Inspect Full History 6
 - 2.4 Restore a Table to an Earlier Version 6
 - 2.5 Understand VACUUM (Do Not Run Carelessly)..... 6
- Lab 3 — Schema Evolution in Practice 6
 - 3.1 Trigger Schema Enforcement (an Expected Failure)..... 6
 - 3.2 Fix It With mergeSchema 7
 - 3.3 Overwrite a Schema Entirely 7
 - 3.4 Rename a Column With Column Mapping 8
- Lab 4 — Databricks Jobs & Scheduling in Practice 8
 - 4.1 Build Three Small Pipeline Notebooks 8
 - 4.2 Create a Job From the Workflows UI 9
 - 4.3 Run It Now and Read the Results..... 10
 - 4.4 Add a Schedule..... 10
 - 4.5 Break It on Purpose to See Retries and Alerts 10
- Capstone — End-to-End Scheduled Pipeline 11
- Troubleshooting Cheat Sheet..... 11

0. Before You Begin

This guide assumes you sign up (or are already signed up) at the Databricks sign-up page and land in Databricks Free Edition — the modern free tier that replaced the older Community Edition in 2025. Free Edition runs on serverless compute that Databricks manages for you, and unlike the legacy Community Edition, it does support the Jobs / Workflows feature used in Lab 4.

If you're on the legacy Community Edition

Labs 1–3 will work exactly as written. For Lab 4, the Workflows/Jobs UI may not be available on legacy Community Edition accounts; if you don't see a “Jobs & Pipelines” or “Workflows” item in the left sidebar, you're on the legacy edition and can still follow along conceptually, or sign up for a new Free Edition account at databricks.com to get the full experience.

Setup (one time)

1. Log into your workspace at the URL you were given at sign-up.
2. In the left sidebar, click New → Notebook.
3. Name it 01_acid_lab. Set the default language to Python. Attach it to Serverless (it should attach automatically — no cluster to configure).
4. Each numbered code block below is one notebook cell. Paste it into a new cell and press Shift+Enter to run it, then compare against the “Expected output” box before moving to the next cell.

A quick note on paths: this guide writes tables to `workspace.default` (a catalog and schema that exist by default on Free Edition) so it works out of the box. If your workspace uses a different catalog/schema name, replace `workspace.default` in the code below with your own catalog and schema (visible under Catalog in the left sidebar).

Lab 1 — ACID Transactions in Practice

1.1 Create Your First Delta Table

This cell creates a small orders table as a managed Delta table — no format option needed, since Delta is the default table format in Databricks.

```
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.default")

spark.sql("""
CREATE OR REPLACE TABLE workspace.default.orders (
  order_id INT,
  customer STRING,
  amount DOUBLE
)
""")
```

```
spark.sql("""
INSERT INTO workspace.default.orders VALUES
  (1, 'Amir', 120.0),
  (2, 'Priya', 75.5),
  (3, 'Jon', 200.0)
""")

display(spark.sql("SELECT * FROM workspace.default.orders ORDER BY order_id"))
```

Expected output

order_id	customer	amount
1	Amir	120.0
2	Priya	75.5
3	Jon	200.0

1.2 Look Inside the Transaction Log

Every Delta table's commit history is queryable directly with DESCRIBE HISTORY — no need to browse the raw files to see what's happened so far.

```
display(spark.sql("DESCRIBE HISTORY workspace.default.orders"))
```

Expected output

version	timestamp	operation	operationParameters
1	<just now>	WRITE	mode=Append
0	<slightly earlier>	CREATE OR REPLACE TABLE	...

Two rows — version 0 for the CREATE OR REPLACE TABLE, and version 1 for the INSERT. Each one is a single atomic commit in `_delta_log`.

1.3 Update the Table and Watch a New Commit Appear

```
spark.sql("UPDATE workspace.default.orders SET amount = amount * 1.1 WHERE customer = 'Jon'")
display(spark.sql("DESCRIBE HISTORY workspace.default.orders").select("version",
"operation", "operationParameters"))
```

Expected output

version	operation	operationParameters
2	UPDATE	predicate=(customer = 'Jon')
1	WRITE	mode=Append
0	CREATE OR REPLACE TABLE	...

Notice the UPDATE didn't edit any file in place — it created version 2. Run `SELECT * FROM workspace.default.orders` again and confirm Jon's amount is now 220.0.

1.4 See Atomicity Protect You From a Failed Write

This cell deliberately runs a malformed INSERT to demonstrate that a failed write commits nothing.

```
try:
  spark.sql("INSERT INTO workspace.default.orders VALUES (4, 'BadRow', 'not_a_number')")
```

```
except Exception as e:
    print("Write failed as expected:", type(e).__name__)

print("Row count after the failed write:", spark.table("workspace.default.orders").count())
```

Expected output

```
Write failed as expected: ParseException (or AnalysisException)
Row count after the failed write: 3
```

The row count is still 3, not 3-and-a-half — the malformed write never partially committed. That's atomicity in action.

Lab 2 — Time Travel in Practice

Create a new notebook `02_time_travel_lab` (or keep using the same one — either works). This lab reuses `workspace.default.orders`.

2.1 Build Up Some Version History

```
spark.sql("INSERT INTO workspace.default.orders VALUES (4, 'Wei', 60.0)")
spark.sql("DELETE FROM workspace.default.orders WHERE customer = 'Priya'")
spark.sql("UPDATE workspace.default.orders SET amount = 999 WHERE customer = 'Amir'")
display(spark.sql("DESCRIBE HISTORY workspace.default.orders").select("version",
"operation"))
```

Expected output

version	operation
5	UPDATE
4	DELETE
3	WRITE
2	UPDATE
1	WRITE
0	CREATE OR REPLACE TABLE

Your version numbers may differ slightly from this example depending on exactly how many cells you ran — that's fine, just note your own current highest version number before continuing.

2.2 Query Past Versions

```
# Replace 1 with the version right after your very first INSERT
display(spark.sql("SELECT * FROM workspace.default.orders VERSION AS OF 1"))
```

Expected output

order_id	customer	amount
1	Amir	120.0
2	Priya	75.5
3	Jon	200.0

This is the table exactly as it looked right after Lab 1, Step 1.2 — before the UPDATE, DELETE, or the second UPDATE ever happened. Amir still shows 120.0, and Priya is still present.

2.3 Inspect Full History

```
hist = spark.sql("DESCRIBE HISTORY workspace.default.orders")
display(hist.select("version", "timestamp", "operation", "operationMetrics"))
```

operationMetrics shows exactly how many rows were added, removed, or updated by each commit — useful for auditing exactly what a given job run did to the table.

2.4 Restore a Table to an Earlier Version

```
# Roll the whole table back to right after Lab 1 (adjust the version number to yours)
spark.sql("RESTORE TABLE workspace.default.orders TO VERSION AS OF 1")
display(spark.sql("SELECT * FROM workspace.default.orders ORDER BY order_id"))
```

Expected output

order_id	customer	amount
1	Amir	120.0
2	Priya	75.5
3	Jon	200.0

(Priya is back, Amir is back to 120.0, Wei is gone)

Run DESCRIBE HISTORY once more — you'll see the RESTORE itself is logged as a brand-new version. Nothing was destroyed; you can even restore forward again to undo the restore if needed.

2.5 Understand VACUUM (Do Not Run Carelessly)

Lab-only demonstration — do not do this to a real table

VACUUM permanently deletes old data files, which breaks time travel to any version that depended on them. The default retention window is 7 days, which is why short retention windows are blocked unless you explicitly disable a safety check. This is shown for awareness only — there's no need to actually run it in this lab.

```
-- DO NOT RUN on a table you care about. Shown for reference only.
-- Default protection blocks short retention windows:
VACUUM workspace.default.orders RETAIN 168 HOURS;  -- 7 days, the safe default

-- Lowering it further is what breaks time travel to recent versions –
-- it requires disabling a safety check first and is NOT recommended here.
```

Lab 3 — Schema Evolution in Practice

3.1 Trigger Schema Enforcement (an Expected Failure)

First, create a fresh table and then try to write a DataFrame that has an extra column the table doesn't expect. This should fail — that failure is schema enforcement doing its job.

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.default.events (
  event_id INT,
  event_type STRING
)
```

```

"""
)

from pyspark.sql import Row
new_events = spark.createDataFrame([
    Row(event_id=1, event_type='click', region='APAC'),
    Row(event_id=2, event_type='purchase', region='EU'),
])

try:
    new_events.write.format("delta").mode("append").saveAsTable("workspace.default.events")
except Exception as e:
    print("Blocked as expected:", type(e).__name__)

```

Expected output

```

Blocked as expected: AnalysisException
(message mentions a schema mismatch / extra column 'region')

```

3.2 Fix It With mergeSchema

```

(new_events.write.format("delta")
    .option("mergeSchema", "true")
    .mode("append")
    .saveAsTable("workspace.default.events"))

display(spark.sql("SELECT * FROM workspace.default.events ORDER BY event_id"))

```

Expected output

event_id	event_type	region
1	click	APAC
2	purchase	EU

Run `DESCRIBE workspace.default.events` (a plain SQL `DESCRIBE`, not `DESCRIBE HISTORY`) and confirm the table now has three columns — `region` was added automatically, and any pre-existing rows would show `NULL` for it.

3.3 Overwrite a Schema Entirely

Now simulate a deliberate migration where a column's meaning changes — for example, replacing `region` with a more granular `country` column.

```

migrated = spark.createDataFrame([
    Row(event_id=1, event_type='click', country='Singapore'),
    Row(event_id=2, event_type='purchase', country='Germany'),
])

(migrated.write.format("delta")
    .option("overwriteSchema", "true")
    .mode("overwrite")
    .saveAsTable("workspace.default.events"))

display(spark.sql("SELECT * FROM workspace.default.events ORDER BY event_id"))

```

Expected output

event_id	event_type	country
1	click	Singapore
2	purchase	Germany

(region column is gone – replaced entirely, as intended)

3.4 Rename a Column With Column Mapping

Column mapping lets you do a true rename — the underlying Parquet files are untouched, only the logical name changes in the log.

```
spark.sql("""
ALTER TABLE workspace.default.events SET TBLPROPERTIES (
  'delta.columnMapping.mode' = 'name',
  'delta.minReaderVersion' = '2',
  'delta.minWriterVersion' = '5'
)
""")

spark.sql("ALTER TABLE workspace.default.events RENAME COLUMN country TO country_name")
display(spark.sql("SELECT * FROM workspace.default.events ORDER BY event_id"))
```

Expected output

event_id	event_type	country_name
1	click	Singapore
2	purchase	Germany

Lab 4 — Databricks Jobs & Scheduling in Practice

This lab builds three tiny notebooks and wires them into a single scheduled job, mirroring the medallion pattern from the main guide.

4.1 Build Three Small Pipeline Notebooks

Create three notebooks with exactly these names, each with the one cell shown.

Notebook: 01_bronze_ingest

```
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.default")
spark.sql("""
CREATE TABLE IF NOT EXISTS workspace.default.sales_bronze (
  order_id INT, customer STRING, amount DOUBLE
)
""")
spark.sql("""
INSERT INTO workspace.default.sales_bronze VALUES
  (101, 'Nina', 45.0), (102, 'Omar', 130.0)
""")
print("Bronze ingest complete")
```

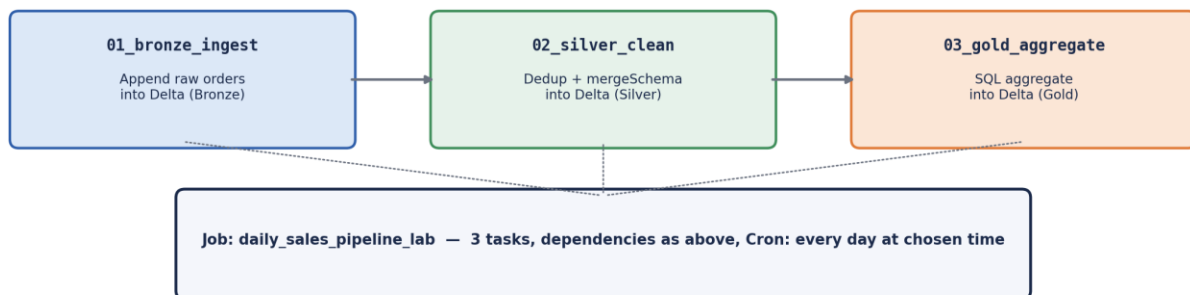
Notebook: 02_silver_clean


```
df = spark.table("workspace.default.sales_bronze").dropDuplicates(["order_id"])
(df.write.format("delta").mode("overwrite")
  .option("mergeSchema", "true")
  .saveAsTable("workspace.default.sales_silver"))
print("Silver clean complete, rows:", df.count())
```

Notebook: 03_gold_aggregate

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.default.sales_gold AS
SELECT customer, SUM(amount) AS total_amount
FROM workspace.default.sales_silver
GROUP BY customer
""")
display(spark.sql("SELECT * FROM workspace.default.sales_gold"))
```

Capstone Lab — Notebooks Wired Into One Scheduled Job



Run manually first with "Run now", confirm success, then add the schedule.

Figure 5. The three notebooks you just wrote, wired together as tasks inside one job.

4.2 Create a Job From the Workflows UI

1. In the left sidebar, click Jobs & Pipelines (this may be labeled Workflows on some accounts).
2. Click Create → Job.
3. Name the job `daily_sales_pipeline_lab`.
4. For the first task, set Task name to `ingest_raw`, Type to Notebook, and Path to your `01_bronze_ingest` notebook. Leave compute as Serverless.
5. Click Add task, name it `clean_transform`, point it at `02_silver_clean`, and under Depends on, select `ingest_raw`.
6. Add a third task `aggregate_kpis` pointing at `03_gold_aggregate`, depending on `clean_transform`.

4.3 Run It Now and Read the Results

1. Click Run now in the top-right corner.
2. Click on the run that appears to open the run details view.

3. Watch the three tasks turn from a queued/gray state to running/blue to succeeded/green in sequence, honoring the dependencies you set.

Expected output

```
ingest_raw:      Succeeded
clean_transform: Succeeded (after ingest_raw)
aggregate_kpis:  Succeeded (after clean_transform)
```

Run `SELECT * FROM workspace.default.sales_gold` in a notebook to confirm Nina and Omar each have a `total_amount` row.

4.4 Add a Schedule

1. From the job's page, click the Schedules & Triggers tab (or the clock icon near the job name).
2. Click Add trigger, choose Scheduled, and pick a simple daily time in your time zone.
3. Save. The job will now run automatically at that time — you don't need to keep the notebook open.

Reminder

On the legacy Community Edition, this Schedules & Triggers tab is not available — jobs can only be run manually there. If that's your situation, running Run now manually still demonstrates the full task-dependency behavior; only the automatic clock trigger is unavailable.

4.5 Break It on Purpose to See Retries and Alerts

Temporarily rename a table reference inside `02_silver_clean` (e.g., change `sales_bronze` to `sales_bronze_typo`), save, and click Run now again.

Expected output

```
ingest_raw:      Succeeded
clean_transform:  Failed — error mentions table or view not found
aggregate_kpis:  Upstream Failed (skipped, since its dependency failed)
```

In the job's Task settings for `clean_transform`, open Retries and set it to retry twice with a short interval — re-run and watch the run history show the retry attempts before the task is finally marked failed. Then open Notifications on the job and add your email under “On failure” so you'd be alerted automatically in a real pipeline. Finally, fix the typo back to `sales_bronze` so the job succeeds again.

Capstone — End-to-End Scheduled Pipeline

Put everything from Labs 1–4 together in one exercise:

1. Confirm `daily_sales_pipeline_lab` from Lab 4 runs successfully end-to-end with Run now.
2. In a scratch notebook, run `DESCRIBE HISTORY workspace.default.sales_gold` and note the current version number.
3. Manually edit `03_gold_aggregate` to intentionally compute the wrong aggregation (e.g., `AVG` instead of `SUM`), save, and click Run now on the job.
4. Confirm `sales_gold` now shows the wrong (averaged) numbers.

5. Use `RESTORE TABLE workspace.default.sales_gold TO VERSION AS OF <the version you noted>` to roll back to the correct data — exactly like fixing a bad production run.
6. Fix `03_gold_aggregate` back to `SUM`, save, and re-run the job to confirm it's now producing correct output going forward.

This is the full loop a real Databricks pipeline goes through: scheduled automation (Jobs), safe concurrent writes (ACID), a graceful way to absorb new fields (schema evolution), and a safety net when something goes wrong in production (time travel).

Troubleshooting Cheat Sheet

Symptom	Likely fix
"Table or view not found: workspace.default..."	Check your catalog/schema name under Catalog in the sidebar and swap it into every path in this guide.
mergeSchema write still fails	Confirm the option is spelled exactly "mergeSchema" and mode is "append" (overwrite needs overwriteSchema instead).
RENAME COLUMN fails with a mapping error	Run the <code>ALTER TABLE ... SET TBLPROPERTIES</code> command from Lab 3.4 first — column mapping must be enabled before a rename.
No "Jobs & Pipelines" / "Workflows" in sidebar	You're likely on the legacy Community Edition; sign up for a new Free Edition workspace to unlock Jobs.
Job task fails with a permissions error on a table	Make sure all three notebooks reference the exact same catalog.schema.table names you created in Lab 4.1.
VERSION AS OF errors with "version not found"	You may have already run <code>VACUUM</code> , or you're referencing a version number higher than the table's current version — re-run <code>DESCRIBE HISTORY</code> to check.

You've now exercised every mechanism from the main guide hands-on: the transaction log, optimistic concurrency, time travel queries and restores, schema enforcement and evolution, and a fully scheduled, dependency-aware Databricks Job. From here, try adapting the capstone pipeline to a dataset of your own.